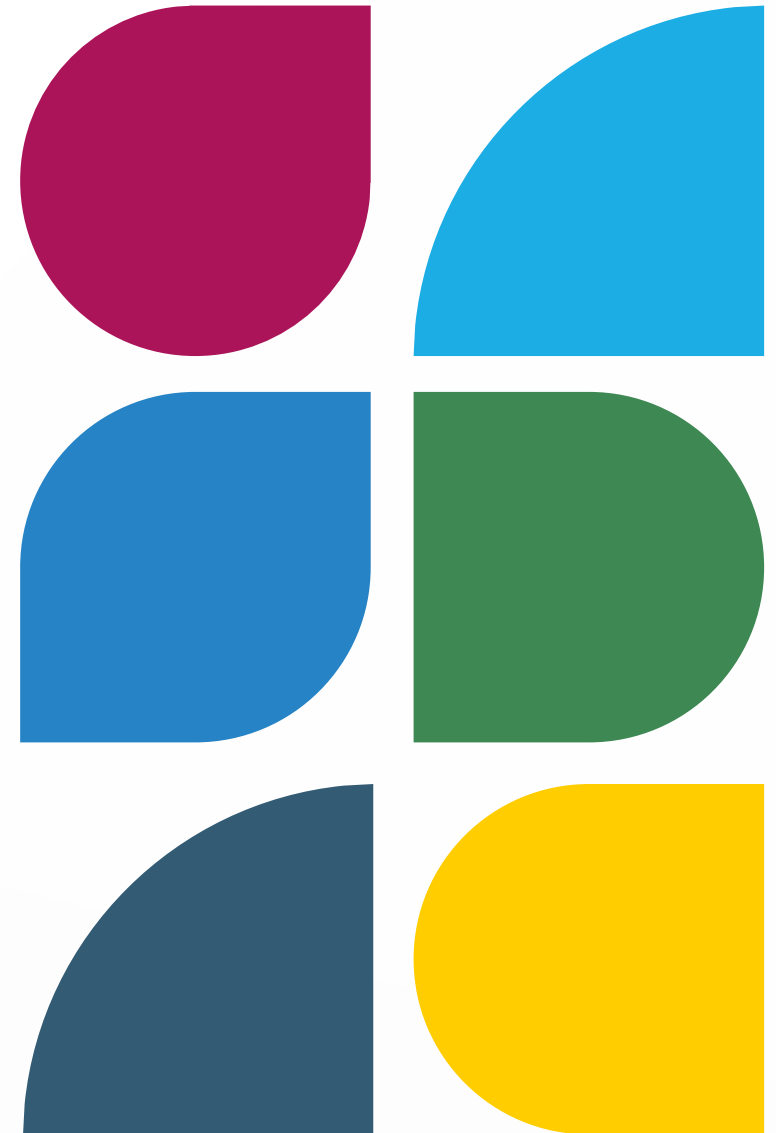
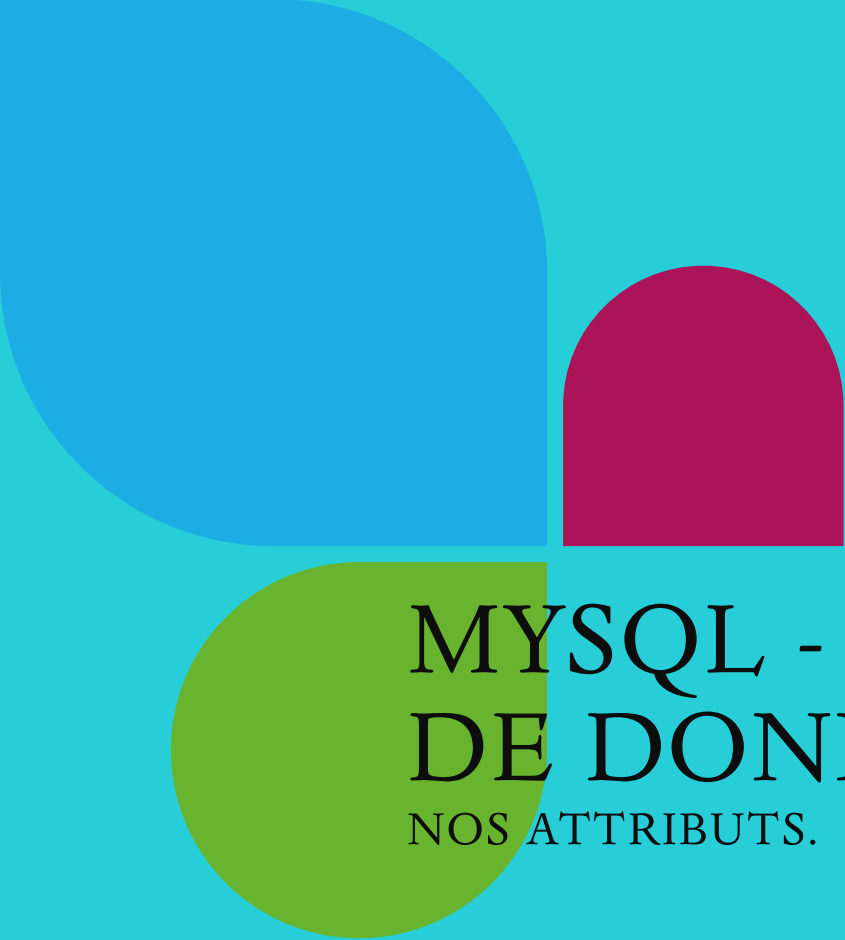


LE LANGAGE SQL STRUCTURED QUERY LANGUAGE

Parcourir et manipuler les données de la base de données.

Concepteur Développeur d'Applications





MYSQL - MANIPULATION DE DONNÉES

NOS ATTRIBUTS.

INSERTION ET MODIFICATION

INSERT

```
INSERT INTO TableName (Column1, Column2, Column3)
VALUES (value1, value2, value3)

/*
Insérer un enregistrement (les valeurs sont insérées
dans l'ordre où colonnes apparaissent dans la base)
*/

INSERT INTO TableName
VALUES (value1, value2, value3)

Deux lignes :

INSERT INTO TableName
VALUES (value1, value2, value3), (value4, value5, value6)

INSERT INTO antiques
VALUES (21, 01, 'Ottoman', 200.00);

INSERT INTO antiques (buyerid, sellerid, item)
VALUES (01, 21, 'Ottoman');
```

UPDATE

```
UPDATE table
SET field1 = newvalue1, field2 = newvalue2
WHERE criteria
ORDER BY field
LIMIT n

/* mise à jour du prix de l'item chair */
UPDATE antiques SET price = 500.00 WHERE item = 'Chair';

/* mise à jour de discount */
UPDATE order SET discount=discount * 1.05
```

SUPPRESSION

DELETE

Utiliser DELETE sans clause WHERE supprime tous les enregistrements.

DELETE informe du nombre de lignes supprimées.

```
DELETE FROM nom_table
WHERE critères;

DELETE t1, t2
FROM table1 t1, table2 t2
WHERE t1.id=t2.id AND t1.value > 1;
```

TRUNCATE

En SQL, la commande TRUNCATE permet de supprimer toutes les données d'une table **sans supprimer la table en elle-même**.

En d'autres mots, cela permet de purger la table.

- Cette instruction diffère de la commande **DROP** qui a pour but de supprimer les données ainsi que la table qui les contient.
- Note : plus rapide et moins consommateur en ressource qu'un DELETE.
- **Réinitialise la valeur de l'auto-incrément.**
- TRUNCATE n'indique pas le nombre de lignes supprimées et pas d'enregistrements dans le journal des modifications.



MYSQL - LES REQUÊTES SÉLECTIONNER LES DONNÉES.

LES REQUÊTES - SELECT

SELECT

La syntaxe de sélection est la suivante :

```
SELECT *  
FROM nom_table  
WHERE condition  
GROUP BY champ1, champ2  
HAVING groupe condition  
ORDER BY champ  
LIMIT limite, taille;
```

Liste de champs

- Il faut spécifier les données à récupérer.
 - * permet d'obtenir tous les champs d'une table.
 - FROM permet de spécifier la table ciblée.
 - Les calculs sont possibles.
 - L'utilisation de fonctions internes à SQL sont possibles.

```
SELECT DATABASE(); -- renvoie le nom de la base courante  
SELECT CURRENT_USER(); -- l'utilisateur courant  
-- renvoie les valeurs du champ "page_id" de la table "Page".  
USE world;  
SELECT page_id FROM `Page`;  
-- idem  
SELECT `world`.`Page`.`page_id`;  
  
SELECT 1+1; -- 2  
SELECT * FROM `Page`;  
SELECT MAX(page_id) FROM `Page`; -- le nombre le plus élevé  
SELECT page_id*2 FROM `Page`; -- le double de chaque identifiant
```

LES REQUÊTES - WHERE ET ORDER BY

WHERE

Cette clause permet de filtrer les enregistrements sur une colonne.

```
SELECT colonne1, colonne2,..., colonneN  
FROM nom_table  
WHERE [condition]
```

- Il est impossible d'utiliser le résultat d'une fonction calculée utilisée d'un SELECT dans le WHERE, car ce résultat n'est trouvé qu'à la fin de l'exécution, donc WHERE ne peut pas s'en servir au moment prévu. Pour ce faire il convient d'utiliser HAVING

ORDER BY

Il est possible de classer les résultats, par ordre croissant ou décroissant, des nombres ou des lettres.

- Par défaut l'ordre est croissant ASC. Pour le décroissant, il faut donc le préciser DESC
- Les valeurs NULL sont considérées comme inférieures aux autres.

```
SELECT liste-colonnes  
FROM nom_table  
[WHERE condition]  
[ORDER BY colonne1, colonne2, .. ] [ASC | DESC];
```

LES REQUÊTES - WHERE

Opérateurs logiques

Opérateur	Symbole	Signification
AND	&&	ET
OR		OU
XOR		OU exclusif
NOT	!	NON

Opérateurs de comparaisons

Opérateur	Signification
=	égal
<	inférieur
<=	inférieur ou égal
>	supérieur
>=	supérieur ou égal
<> ou !=	différent
<=>	égal (valable pour NULL aussi)

SELECT .. INTO

Le SELECT ... INTO permet de stocker un résultat de requête dans des variables

- Le positionnement de INTO est préféré en fin de la requête soit après le FROM depuis la version 8.0.20 de MySQL

Documentation officielle :

<https://dev.mysql.com/doc/refman/8.0/en/select-into.html>

Exemple :

```
-- compter le nombre d'emprunts en cours de l'adhérent
-- select .. from t into @var;
SELECT COUNT(*)
FROM pret
WHERE numab = pNumab AND dateretour IS NULL
INTO nb_emprunts ;
```

LIKE ET LES JOKERS

Si l'on cherche un mot particulier ?

L'opérateur **LIKE** est très utile, car il permet de faire des recherches en utilisant des "jokers".

Deux jokers existent pour LIKE :

% : qui représente n'importe quelle chaîne de caractères, quelle que soit sa longueur (y compris une chaîne de longueur 0) ;

_ : qui représente un seul caractère.

- Quelques exemples :
 - 'b%' cherchera toutes les chaînes de caractères commençant par 'b' ("brocoli", "bouli", "b").
 - 'b_' cherchera toutes les chaînes de caractères contenant deux lettres dont la première est 'b' ("ba", "bf", "b8").
- Comment faire si vous cherchez une chaîne de caractères contenant % ou _ ?
 - Il faut donc signaler à MySQL que vous ne désirez pas utiliser % ou _ en tant que joker, mais bien en tant que caractère de recherche.
 - Pour ça, il suffit de mettre le caractère d'échappement \, dont je vous ai déjà parlé, devant le % ou le _.

```
SELECT *  
FROM Animal  
WHERE commentaires LIKE '%\%\%';
```

LES REQUÊTES - LIMIT ET DISTINCT

LIMIT :

Le nombre maximum d'enregistrements dans le résultat est facultatif, on l'indique avec le mot LIMIT.

Généralement, cela s'emploie après un ORDER BY pour avoir les maximums et minimums.

DISTINCT :

Le mot DISTINCT peut être utilisé pour supprimer les doublons des lignes du résultat.

```
/* Ce résultat retourne donc entre 0 et 10 lignes. */
SELECT * FROM `Page` ORDER BY page_id LIMIT 10;
/* Ce résultat retourne donc entre 0 et 30 lignes. */
SELECT * FROM `Page` ORDER BY rand() LIMIT 3;
/* Possible de définir une plage d'enregistrements, sachant que le premier est
le numéro zéro */
SELECT * FROM `Page` ORDER BY page_id LIMIT 10;
SELECT * FROM `Page` ORDER BY page_id LIMIT 0, 10; -- synonyme
/* On peut donc paginer les requêtes dont les résultats peuvent saturer le
serveur */
SELECT * FROM `Page` ORDER BY page_id LIMIT 0, 10; -- première page
SELECT * FROM `Page` ORDER BY page_id LIMIT 10, 10; -- seconde page
SELECT * FROM `Page` ORDER BY page_id LIMIT 20, 10; -- troisième page

/* DISTINCT */
SELECT DISTINCT * FROM `Page`      -- aucun doublon
SELECT DISTINCTROW * FROM `Page`  -- synonyme
SELECT ALL * FROM `Page`          -- doublons (comportement par défaut)

/* Récupérer la liste de toutes les valeurs différentes d'un champ */
SELECT DISTINCT `user_real_name` FROM `Page` ORDER BY `user_real_name`

/* Sortir les différentes combinaisons de valeurs */
SELECT DISTINCT `user_real_name`, `user_editcount` FROM `Page`
ORDER BY `user_real_name`
```

IN ET NOT IN

L'opérateur logique IN dans SQL s'utilise avec la commande WHERE pour vérifier si une colonne est égale à une des valeurs comprises dans un ensemble de valeurs déterminés.

C'est une méthode simple pour vérifier si une colonne est égale à une valeur OU une autre valeur OU une autre valeur et ainsi de suite, sans avoir à utiliser de multiple fois l'opérateur OR

```
SELECT nom_colonne
FROM table
WHERE nom_colonne IN ( valeur1, valeur2, valeur3, ... )
```

Exemple

Imaginons une table "adresse" qui contient une liste d'adresse associée à des utilisateurs d'une application.

id	id_utilisateur	addr_rue	addr_code_postal	addr_ville
1	23	35 Rue Madeleine Pelletier	25250	Bournois
2	43	21 Rue du Moulin Collet	75006	Paris
3	65	28 Avenue de Cornouaille	27220	Mousseaux-Neuville
4	67	41 Rue Marcel de la Provoté	76430	Graimbouville
5	68	18 Avenue de Navarre	75009	Paris

Si l'on souhaite obtenir les enregistrements des adresses de Paris et de Graimbouville, il est possible d'utiliser la requête suivante:

```
SELECT *
FROM adresse
WHERE addr_ville IN ( 'Paris', 'Graimbouville' )
```

Résultats :

id	id_utilisateur	addr_rue	addr_code_postal	addr_ville
2	43	21 Rue du Moulin Collet	75006	Paris
4	67	41 Rue Marcel de la Provoté	76430	Graimbouville
5	68	18 Avenue de Navarre	75009	Paris

BETWEEN

L'opérateur BETWEEN est utilisé dans une requête SQL pour sélectionner un intervalle de données dans une requête utilisant WHERE.

L'intervalle peut être constitué de chaînes de caractères, de nombres ou de dates.

L'exemple le plus concret consiste par exemple à récupérer uniquement les enregistrements entre 2 dates définies.

```
SELECT *  
FROM table  
WHERE nom_colonne BETWEEN 'valeur1' AND 'valeur2'
```

Exemple : filtrer entre 2 dates

Imaginons une table "utilisateur" qui contient les membres d'une application en ligne.

id	nom	date_inscription
1	Maurice	2012-03-02
2	Simon	2012-03-05
3	Chloé	2012-04-14
4	Marie	2012-04-15
5	Clémentine	2012-04-26

Si l'on souhaite obtenir les membres qui se sont inscrits entre le 1 avril 2012 et le 20 avril 2012 il est possible d'effectuer la requête suivante:

```
SELECT *  
FROM utilisateur  
WHERE date_inscription BETWEEN '2012-04-01' AND '2012-04-20'
```

Résultat :

id	nom	date_inscription
3	Chloé	2012-04-14
4	Marie	2012-04-15

UNION

La clause/opérateur SQL UNION est utilisée pour combiner les résultats de deux ou plusieurs instructions SELECT sans renvoyer les lignes en double.

Pour utiliser cette clause UNION, chaque instruction SELECT doit avoir :

- Le même nombre de colonnes sélectionnées
- Le même type de données et
- Les avoir dans le même ordre

Mais ils n'ont pas besoin d'être dans la même longueur.

La clause UNION produit des valeurs distinctes dans le jeu de résultats. Pour extraire les valeurs en double, UNION ALL doit être utilisée à la place de UNION.

Syntaxe - UNION

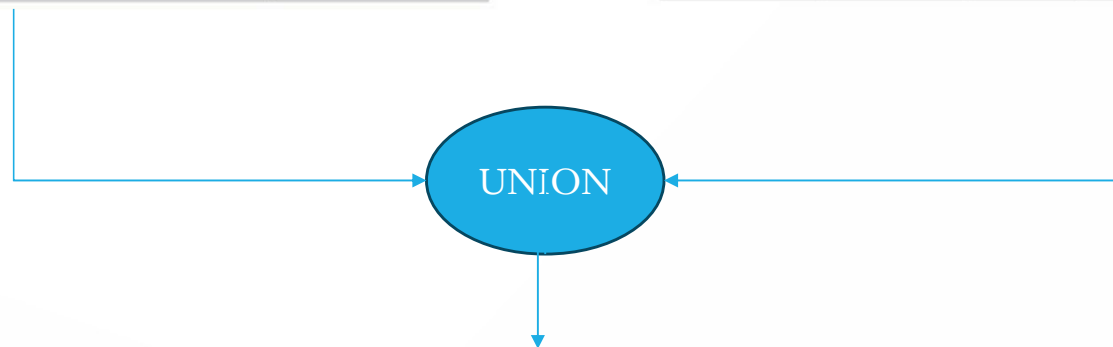
```
1 SELECT colonne1 [, colonne2 ]
2 FROM table1 [, table2 ]
3 [WHERE condition]
4
5 UNION
6
7 SELECT colonne1 [, colonne2 ]
8 FROM table1 [, table2 ]
9 [WHERE condition]
```

Syntaxe - UNION ALL

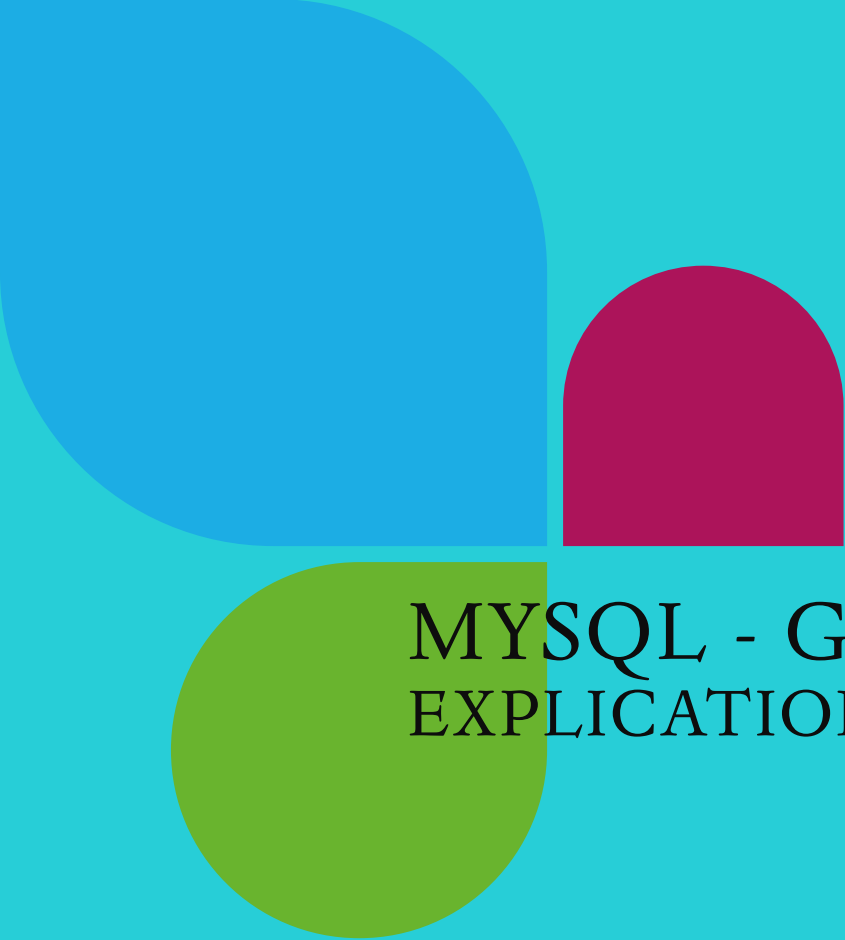
```
1 SELECT colonne1 [, colonne2 ]
2 FROM table1 [, table2 ]
3 [WHERE condition]
4
5 UNION ALL
6
7 SELECT colonne1 [, colonne2 ]
8 FROM table1 [, table2 ]
9 [WHERE condition]
```

prenom	nom	ville	date_naissance	total_achat
Léon	Dupuis	Paris	1983-03-06	135
Marie	Bernard	Paris	1993-07-03	75
Sophie	Dupond	Marseille	1986-02-22	27
Marcel	Martin	Paris	1976-11-24	39

prenom	nom	ville	date_naissance	total_achat
Marion	Leroy	Lyon	1982-10-27	285
Paul	Moreau	Lyon	1976-04-19	133
Marie	Bernard	Paris	1993-07-03	75
Marcel	Martin	Paris	1976-11-24	39



prenom	nom	ville	date_naissance	total_achat
Léon	Dupuis	Paris	1983-03-06	135
Marie	Bernard	Paris	1993-07-03	75
Sophie	Dupond	Marseille	1986-02-22	27
Marcel	Martin	Paris	1976-11-24	39
Marie	Leroy	Lyon	1982-10-27	285
Paul	Moreau	Lyon	1976-04-19	133



MYSQL - GROUP BY ET HAVING EXPLICATIONS ET DÉMONSTRATION

ORGANISER DES DONNÉES IDENTIQUES EN GROUPES - GROUP BY ET HAVING

La clause GROUP BY en SQL permet d'organiser des données identiques en groupes à l'aide de certaines fonctions.

C'est-à-dire si une colonne particulière a les mêmes valeurs dans différentes lignes, elle organisera ces lignes dans un groupe.

- La clause GROUP BY est utilisée avec l'instruction SELECT.
- Dans la requête, la clause GROUP BY est placée après la clause WHERE.
- Dans la requête, la clause GROUP BY est placée avant la clause ORDER BY si elle est utilisée.

- Vous pouvez également utiliser certaines fonctions d'agrégation telles que COUNT, SUM, MIN, MAX, AVG, etc. sur la colonne groupée.

```
SELECT colonne1, colonne2, ... colonneN,  
fonction_agregation (nom_colonne)  
FROM tables  
[WHERE conditions]  
GROUP BY colonne1, colonne2, ... colonneN;
```

- *colonne1, colonne2, ... colonneN - spécifie les colonnes(ou expressions) qui ne sont pas encapsulées dans une fonction d'agrégation et doivent être incluses dans la clause GROUP BY.*

```
SELECT AGE FROM  
EMPLOYES GROUP BY  
AGE;
```

Age
25
30
29
30
30
29



Age
25
29
30

Cette requête nous renvoie 3 résultats soit 3 groupes :

- Groupe 1 - Age = 25

Age
25

- Groupe 2 - Age = 29

Age
29
29

- Groupe 3 - Age = 30

Age
30
30
30

FONCTION D'AGRÉGATIONS

Vous pouvez compter désormais le nombre d'employés de chaque groupe à l'aide de count :

```
SELECT Age, count(*) AS "Nombre d'employés"  
FROM Employes  
GROUP BY Age;
```

Age	Nombre d'employés
25	1
29	2
30	3

Fonction	Description
COUNT()	Compte le nombre de lignes (ou valeurs non nulles si une colonne est spécifiée).
SUM()	Calcule la somme des valeurs numériques.
AVG()	Calcule la moyenne des valeurs numériques.
MIN()	Renvoie la plus petite valeur.
MAX()	Renvoie la plus grande valeur.

HAVING

Nous savons que la clause WHERE est utilisée pour imposer des conditions aux colonnes, mais que se passe-t-il si nous voulons imposer des conditions aux groupes ?

- C'est ici que la clause **HAVING** entre en vigueur.

Nous pouvons utiliser la clause HAVING pour poser des conditions afin de décider quel groupe fera partie de l'ensemble des résultats finaux.

- De plus, nous ne pouvons pas utiliser les fonctions d'agrégation telles que SUM(), COUNT(), etc. avec la clause WHERE.

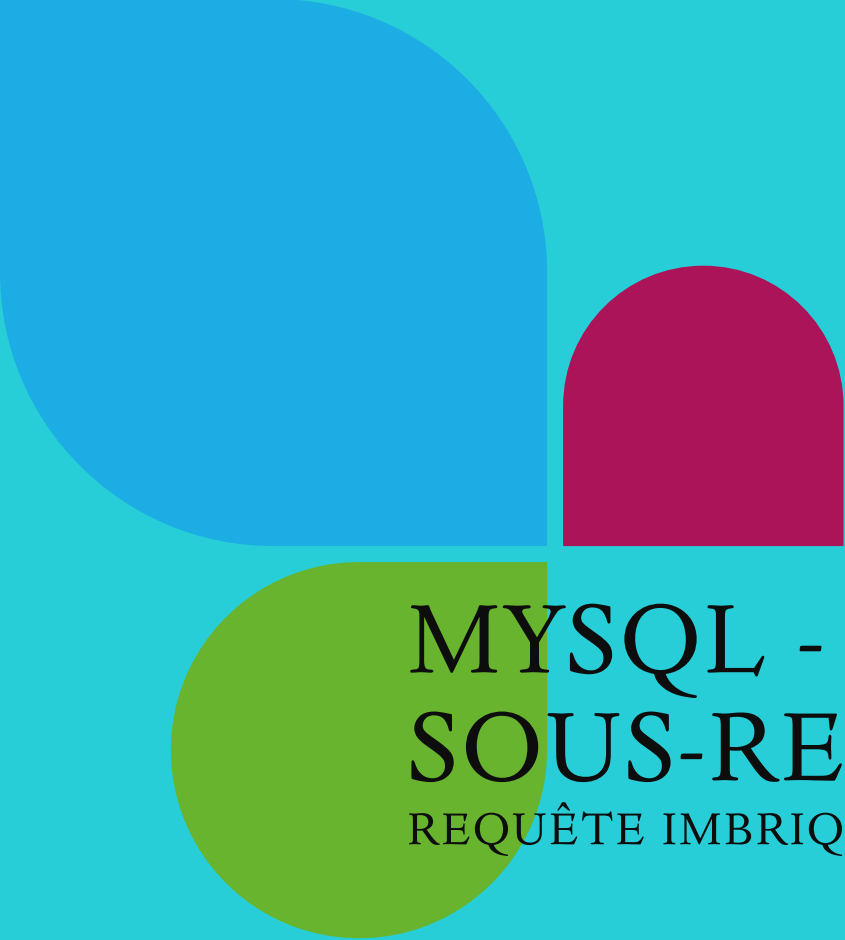
Nous devons donc utiliser la clause HAVING si nous voulons utiliser l'une de ces fonctions dans les conditions.

La requête suivante récupère les noms de département et le salaire moyen de chaque département :

```
SELECT D.Nom_dep, AVG(E.Salaire) AS "Salaire moyen"
FROM Employes AS E INNER JOIN Departement AS D
ON E.Dep=D.Id_dep
GROUP BY D.Nom_dep;
```

Supposons maintenant que nous ne voulions montrer que les départements dont le salaire moyen est supérieur à 6000 ?

```
SELECT D.Nom_dep, AVG(E.Salaire) AS "Salaire moyen"
FROM Employes AS E INNER JOIN Departement AS D
ON E.Dep=D.Id_dep
GROUP BY D.Nom_dep
HAVING AVG(E.Salaire) > 6000;
```



MYSQL - REQUÊTE AVEC SOUS-REQUÊTE

REQUÊTE IMBRIQUÉE

LES SOUS-REQUÊTES EN SQL

Une sous-requête, également appelée requête imbriquée ou sous-sélection, est une requête SELECT intégrée à la clause WHERE ou HAVING d'une autre requête SQL.

Les données renvoyées par la sous-requête sont utilisées par l'instruction externe de la même manière qu'une valeur littérale serait utilisée.

Les sous-requêtes constituent un moyen simple et efficace de gérer les requêtes qui dépendent des résultats d'une autre requête.

Sous-requêtes vs jointures :

Comparées aux jointures, les sous-requêtes sont simples à utiliser et à lire. Ils ne sont pas aussi compliqués que les jointures.

Mais les sous-requêtes posent des problèmes de performances.

[L'utilisation d'une jointure au lieu d'une sous-requête peut parfois vous donner un gain de performances jusqu'à 500 fois.](#)

Si vous avez le choix, il est recommandé d'utiliser une jointure plutôt qu'une sous-requête.

```
SELECT nom_colonne [, nom_colonne ]  
FROM   table1 [, table2 ]  
WHERE  nom_colonne OPERATOR  
       (SELECT nom_colonne [, nom_colonne ]  
        FROM table1 [, table2 ]  
        [WHERE])
```

EXIST ET NOT EXISTS

Les conditions EXISTS et NOT EXISTS s'utilisent de la manière suivante :

```
SELECT * FROM nom_table  
WHERE [NOT] EXISTS (sous-requête)
```

Une condition avec EXISTS sera vraie (et donc la requête renverra quelque chose) si la sous-requête correspondante renvoie au moins une ligne.

Une condition avec NOT EXISTS sera vraie si la sous-requête correspondante ne renvoie aucune ligne.

Table commande :

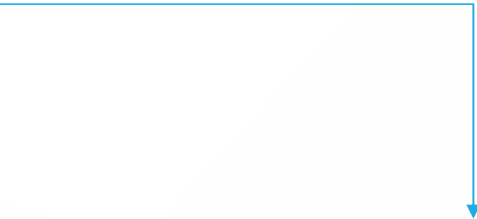
c_id	c_date_achat	c_produit_id	c_quantite_produit
1	2014-01-08	2	1
2	2014-01-24	3	2
3	2014-02-14	8	1
4	2014-03-23	10	1

Table produit :

p_id	p_nom	p_date_ajout	p_prix
2	Ordinateur	2013-11-17	799.9
3	Clavier	2013-11-27	49.9
4	Souris	2013-12-04	15
5	Ecran	2013-12-15	250

RÉSULTAT

```
SELECT *  
FROM commande  
WHERE EXISTS (  
    SELECT *  
    FROM produit  
    WHERE c_produit_id = p_id  
)
```



c_id	c_date_achat	c_produit_id	c_quantite_produit
1	2014-01-08	2	1
2	2014-01-24	3	2

ALL

Dans le langage SQL, la commande ALL permet de comparer une valeur dans l'ensemble de valeurs d'une sous-requête.

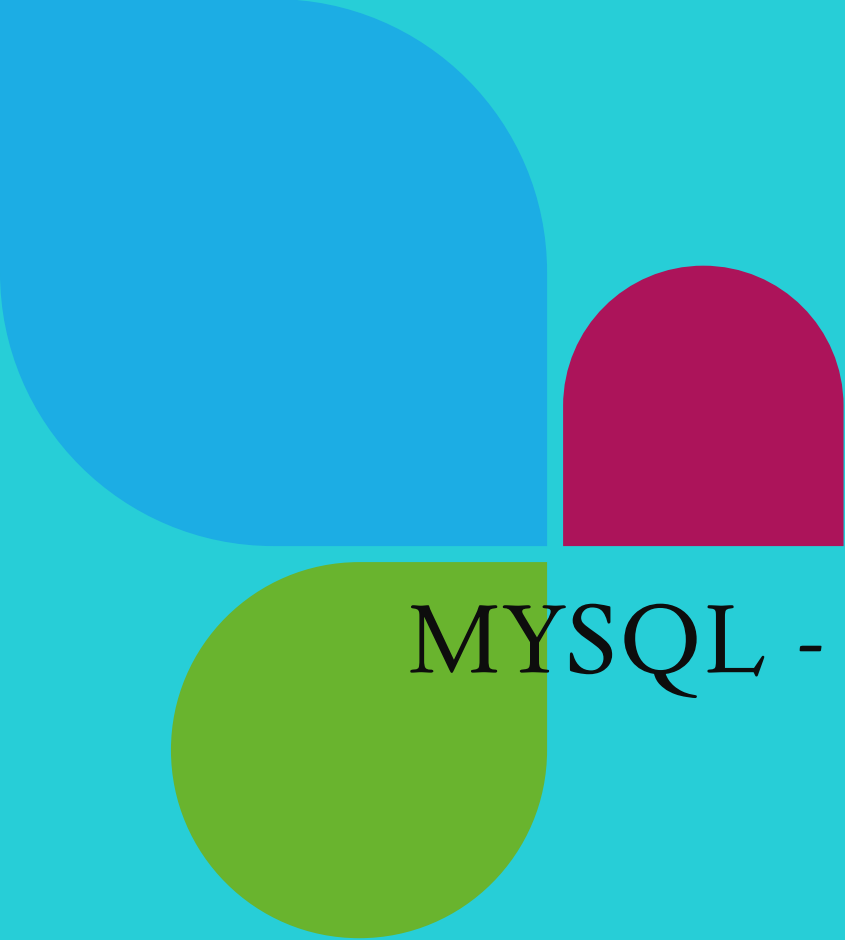
En d'autres mots, cette commande permet de s'assurer qu'une condition est "égale", "différente", "supérieure", "inférieure", "supérieure ou égale" ou "inférieure ou égale" pour tous les résultats retournés par une sous-requête.

Imaginons une requête :

```
SELECT colonne1
FROM table1
WHERE colonne1 > ALL (
    SELECT colonne1
    FROM table2
)
```

Avec cette requête, si nous supposons que dans table1 il y a un résultat avec la valeur 10, voici les différents résultats de la condition selon le contenu de table2 :

- La condition est vraie si table2 contient {-5,0,+5} car toutes les valeurs sont inférieures à 10
- La condition est fausse si table2 contient {12,6,NULL,-100} car au moins une valeur est inférieure à 10
- La condition est non connue si table2 est vide



MYSQL - LES VUES

LES VUES

Les vues sont des objets de la base de données, constitués d'un **nom** et d'une **requête de sélection**.

La requête SELECT stockée dans une vue :

- peut utiliser des jointures, des clauses WHERE, GROUP BY, des fonctions (scalaires ou d'agrégation), etc.
- **L'utilisation de DISTINCT et LIMIT est cependant déconseillée.**

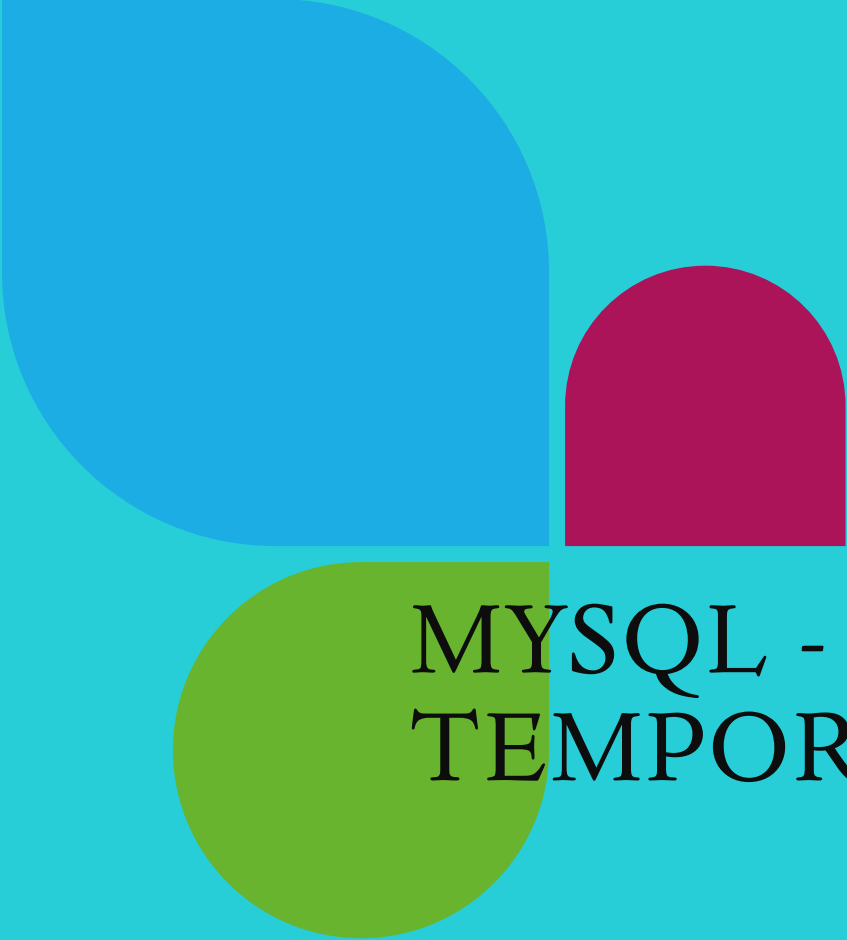
On peut sélectionner les données à partir d'une vue de la même manière qu'on le fait à partir d'une table. On peut donc utiliser des jointures, des fonctions, des GROUP BY, des LIMIT...

Les vues permettent de simplifier les requêtes, de créer une interface entre l'application et la base de données, et/ou de restreindre finement l'accès en lecture des données aux utilisateurs.

```
-- Syntaxe
CREATE [OR REPLACE] VIEW nom_vue
AS requete_select;

-- Creation d'une vue V_Chien_Race
CREATE OR REPLACE VIEW V_Chien_race
AS SELECT id, sexe, date_naissance, nom, commentaires, espece_id, race_id,
mere_id, pere_id, disponible
FROM Chien
WHERE race_id IS NOT NULL;

-- Consultation
SELECT * FROM V_Chien_Race;
```



MYSQL - TABLES TEMPORAIRES

TABLES TEMPORAIRES

Une table temporaire n'existe que pour la session dans laquelle elle a été créée.

- Dès que la session se termine, les tables temporaires sont supprimées.

Objectifs :

1. Elles permettent de gagner en performance lorsque dans une session, on doit exécuter plusieurs requêtes sur un même set de données.
2. On peut les utiliser pour créer des données de test.
3. On peut les utiliser pour stocker un ensemble de résultats d'une procédure stockée.

- créée de la même manière qu'une table normale. Il suffit d'ajouter le mot-clé **TEMPORARY** avant **TABLE**.

```
CREATE TEMPORARY TABLE name (  
  
);
```

- créer une table (temporaire ou non) à partir de la structure d'une autre table :

```
CREATE [TEMPORARY] TABLE newTable LIKE oldTable;
```

- créer une table à partir d'une requête SELECT :

```
CREATE [TEMPORARY] TABLE newTable AS  
SELECT ...  
FROM ...  
WHERE ...;
```



MYSQL - LES JOINTURES

PARCOURIR UNE BASE DE DONNÉES À LA RECHERCHE DE RÉSULTATS.

PRINCIPE

On souhaite obtenir le nom latin de l'animal nommé Cartouche. Comment faire ?

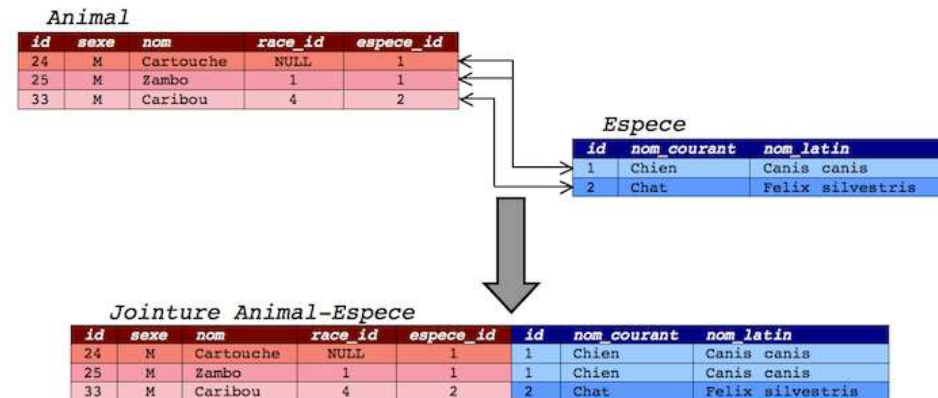
Un select sur la table Animal avec comme clause le nom "Cartouche" nous donne en résultat la ligne de l'id 24.

Un select sur la table Espece contient le nom latin..

Le point commun entre nos deux tables est :

- Espece_id de la table Animal
- Id de la table Espece

Ce qui nous donnerai alors une nouvelle table



Principe des jointures

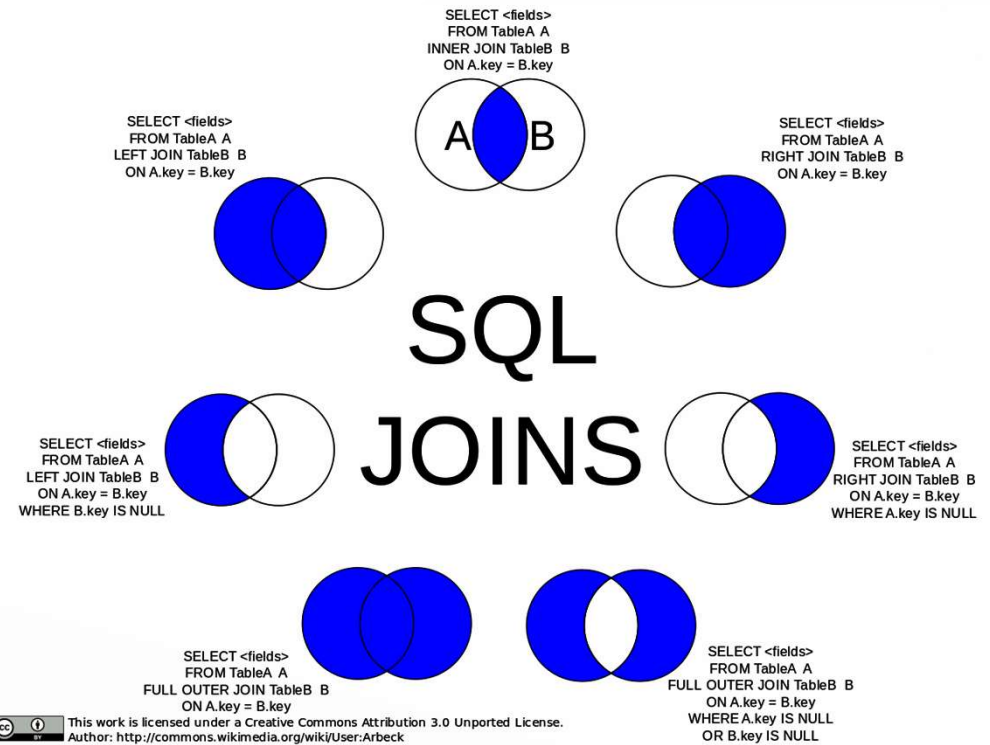
On va donc utiliser une jointure entre nos deux tables :

```
SELECT Espece.nom_latin
FROM Espece
INNER JOIN Animal
ON Espece.id = Animal.espece_id
WHERE Animal.nom = 'Cartouche';
```

DIFFÉRENTES JOINTURES

Il existe plusieurs types de jointures en SQL

La jointure par défaut est la INNER JOIN (ou JOIN)

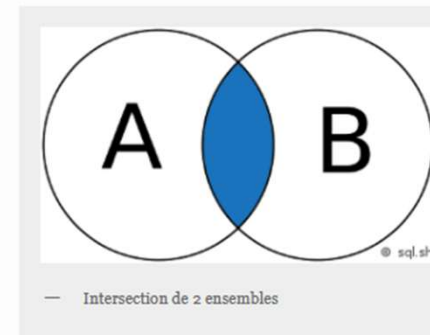


INTERNE

La jointure interne indique que l'on exige qu'il y ait des données de part et d'autre de la jointure. Donc si on fait une jointure sur la colonne a de la table A et la colonne b de la table B :

Cela retournera uniquement les lignes pour lesquelles A.a et B.b correspondent.

INNER JOIN



```
SELECT *  
FROM A  
INNER JOIN B ON A.key = B.key
```

EXTERNE

Une jointure externe permet de sélectionner également les lignes pour lesquelles il n'y a pas de correspondance dans une des tables jointes.

Deux types de jointures : par la gauche ou par la droite.

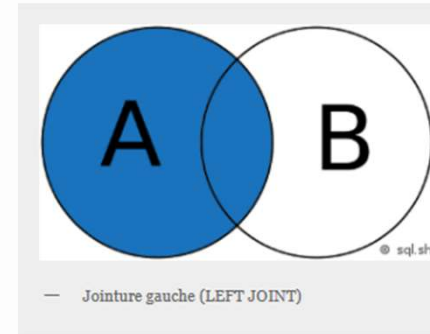
Jointures par la gauche LEFT JOIN ou LEFT OUTER JOIN

On veut toutes les lignes de la table de gauche (sauf restrictions dans une clause WHERE), même si certaines n'ont pas de correspondance avec une ligne de la table de droite.

Jointures par la droite RIGHT JOIN ou RIGHT OUTER JOIN

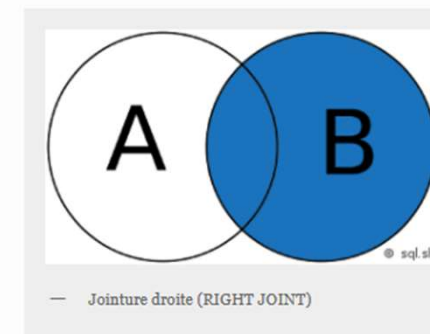
On veut toutes les lignes de la table de droite (sauf restrictions dans une clause WHERE), même si certaines n'ont pas de correspondance avec une ligne de la table de gauche.

LEFT JOIN



```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key
```

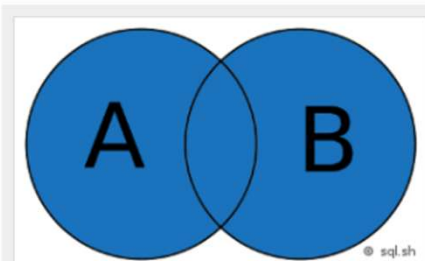
RIGHT JOIN



```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key
```

FULL

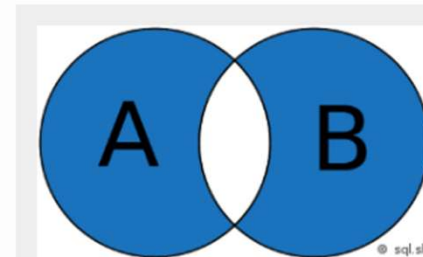
FULL JOIN



— Union de 2 ensembles

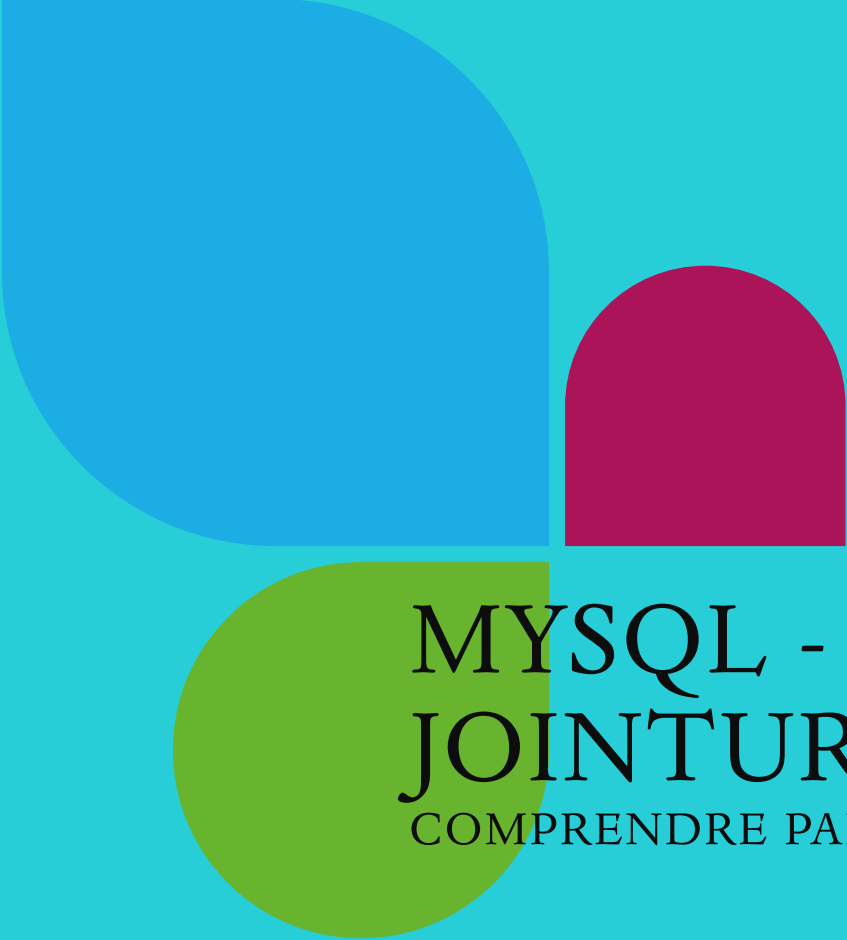
```
SELECT *
FROM A
FULL JOIN B ON A.key = B.key
```

FULL JOIN (sans intersection)



— Jointure pleine (FULL JOIN sans intersection)

```
SELECT *
FROM A
FULL JOIN B ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL
```



MYSQL - EXEMPLE DE JOINTURES

COMPRENDRE PAR L'EXEMPLE

ENONCÉ

Soit la table Utilisateur

id	prenom	nom	email	ville
1	Aimée	Marechal	aime.marechal@example.com	Paris
2	Esmée	Lefort	esmee.lefort@example.com	Lyon
3	Marine	Prevost	m.prevost@example.com	Lille
4	Luc	Rolland	lucrolland@example.com	Marseille

Soit la table Commande

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
2	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

INNER JOIN

Pour afficher toutes les commandes associées aux utilisateurs, il est possible d'utiliser la requête suivante :

```
-- inner join
SELECT id, prenom, nom, date_achat, num_facture, prix_total
FROM utilisateur
INNER JOIN commande ON utilisateur.id = commande.utilisateur_id;
```

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35

Le résultat de la requête montre la jointure entre les 2 tables. Les utilisateurs 3 et 4 ne sont pas affichés puisqu'il n'y a pas de commandes associés à ces utilisateurs.

L'utilisateur 5 n'étant pas existant, n'est donc pas affiché puisque la condition n'est pas vrai entre les 2 tables.

LEFT JOIN

Pour lister tous les utilisateurs avec leurs commandes et afficher également les utilisateurs qui n'ont pas effectués d'achats, il est possible d'utiliser la requête suivante :

```
-- left join
SELECT *
FROM utilisateur
LEFT JOIN commande ON utilisateur.id = commande.utilisateur_id;
```

Si on veut simplement filtrer que les utilisateurs ayant effectués des achats :

```
-- left join avec filtrage
SELECT id, prenom, nom, utilisateur_id
FROM utilisateur
LEFT JOIN commande ON utilisateur.id = commande.utilisateur_id
WHERE utilisateur_id IS NULL;
```

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35
3	Marine	Prevost	NULL	NULL	NULL
4	Luc	Rolland	NULL	NULL	NULL

Les dernières lignes montrent des utilisateurs qui n'ont effectuée aucune commandes. La ligne retourne la valeur NULL pour les colonnes concernant les achats qu'ils n'ont pas effectués.

RIGHT JOIN

Pour afficher toutes les commandes avec le nom de l'utilisateur correspondant il est normalement d'habitude d'utiliser INNER JOIN en SQL. Malheureusement, si l'utilisateur a été supprimé de la table, alors ça ne retourne pas l'achat. L'utilisation de RIGHT JOIN permet de retourner tous les achats et d'afficher le nom de l'utilisateur s'il existe.

Pour cela il convient d'utiliser cette requête:

```
-- right join
SELECT id, prenom, nom, utilisateur_id, date_achat, num_facture
FROM utilisateur
RIGHT JOIN commande ON utilisateur.id = commande.utilisateur_id;
```

id	prenom	nom	utilisateur_id	date_achat	num_facture
1	Aimée	Marechal	1	2013-01-23	A00103
1	Aimée	Marechal	1	2013-02-14	A00104
2	Esmée	Lefort	2	2013-02-17	A00105
3	Marine	Prevost	3	2013-02-21	A00106
NUL L	NULL	NULL	5	2013-03-02	A00107

*Ce résultat montre que la facture **A00107** est liée à l'utilisateur numéro 5.*

Or, cet utilisateur n'existe pas ou n'existe plus.

Grâce à RIGHT JOIN, l'achat est tout de même affiché mais les informations liées à l'utilisateur sont remplacé par NULL.

FULL JOIN

Il est possible d'utiliser FULL JOIN pour lister tous les utilisateurs ayant effectué ou non une vente, et de lister toutes les ventes qui sont associées ou non à un utilisateur.

La requête SQL est la suivante :

```
-- full join
SELECT id, prenom, nom, utilisateur_id, date_achat, num_facture
FROM utilisateur
FULL JOIN commande ON utilisateur.id = commande.utilisateur_id;
```

id	prenom	nom	utilisateur_id	date_achat	num_facture
1	Aimée	Marechal	1	2013-01-23	A00103
1	Aimée	Marechal	1	2013-02-14	A00104
2	Esmée	Lefort	2	2013-02-17	A00105
3	Marine	Prevost	3	2013-02-21	A00106
4	Luc	Rolland	NULL	NULL	NULL
NULL	NULL	NULL	5	2013-03-02	A00107

Ce résultat affiche bien l'utilisateur numéro 4 qui n'a effectué aucun achat.

Le résultat retourne également la facture **A00107** qui est associée à un utilisateur qui n'existe pas (ou qui n'existe plus).

Dans les cas où il n'y a pas de correspondance avec l'autre table, les valeurs des colonnes valent NULL.

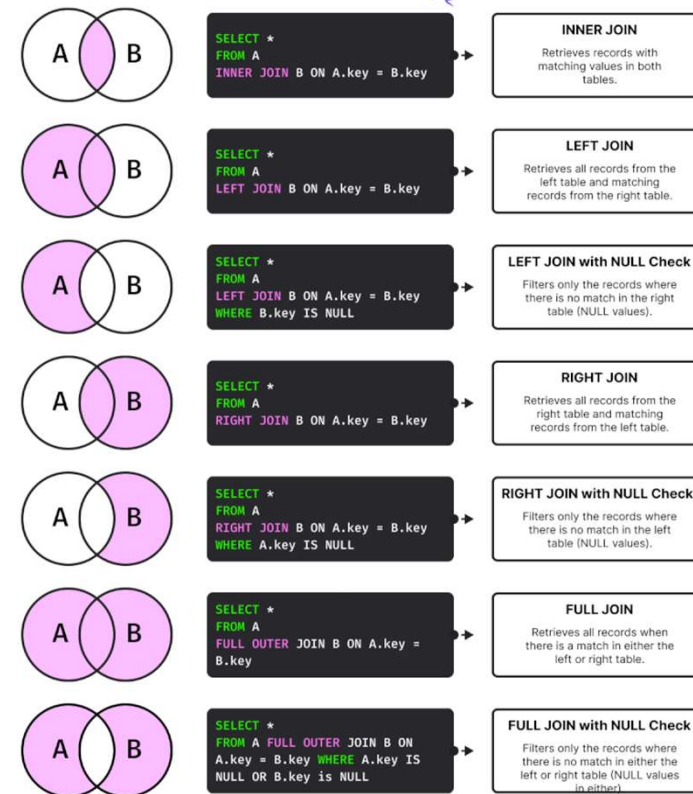
RÉSUMÉ

Un petit récapitulatif

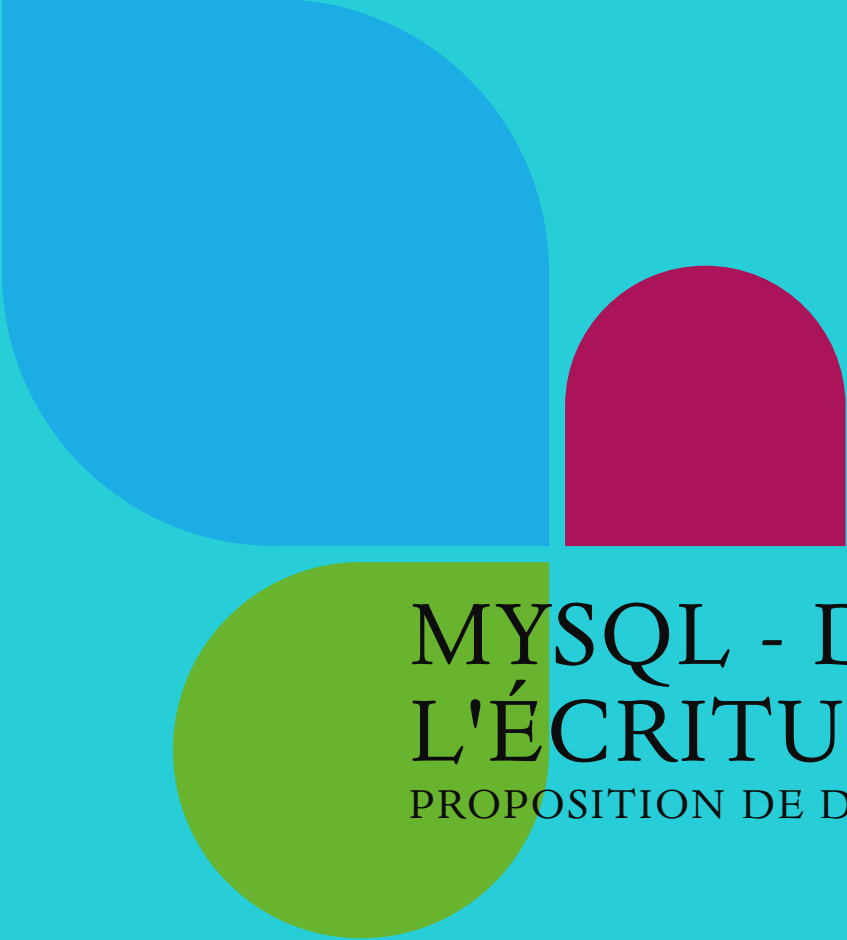


Finalement, c'est facile...

SQL JOINS



amigoscode.com



MYSQL - DÉMARCHE POUR L'ÉCRITURE DE SELECT

PROPOSITION DE DÉMARCHE

QUELQUES CONSEILS

1. Décider quels sont les attributs à visualiser, les inclure dans la clause **SELECT**.
2. Les expressions présentes dans la liste de sélection d'une requête (clause **SELECT**) avec la clause **GROUP BY** doivent être des fonctions d'agrégation ou apparaître dans la liste **GROUP BY**.
3. Déterminer les tables à mettre en jeu, les inclure dans la clause **FROM**.
4. Déterminer les conditions de jointure quand plusieurs tables sont en jeu.
5. Déterminer les conditions limitant la recherche : les conditions portant sur les groupes doivent figurer dans une clause **HAVING**, celles portant sur des valeurs individuelles dans une clause **WHERE**.
6. Préciser l'ordre d'apparition des lignes de résultat dans une clause **ORDER BY**.

On peut formaliser la démarche en remplissant le tableau suivant pour aider à construire chaque requête :

Questions ?	Réponse par la requête
Quelles sont les colonnes à afficher en résultat (SELECT)	SELECT ...
De quelles tables sont issues ces colonnes (FROM)	FROM ...
Y'a-t-il des jointures (JOIN)	...
N'y-a-t-il que certaines lignes à prendre en compte (WHERE)	WHERE ...
Veut-on un résultat par "paquets" de lignes (GROUP BY)	GROUP BY ...
Veut-on voir apparaître les résultats selon un ordre précis ? (ORDER BY)	ORDER BY ...
N'y-a-t-il que certaines lignes résultats à prendre en compte ? (HAVING)	HAVING ...



EXERCICES : MANIPULER SQL POUR SÉLECTIONNER DES DONNÉES

EXERCICES

Réaliser les exercices du document :

1. CDA24060_MySQL presentiel_Exos.pdf
2. Utiliser les scripts fournis pour chaque exercice dans MySQL Workbench
3. Réaliser un script par exercice regroupant vos requêtes pour répondre aux questions posées.

